

Lecture 12: Control flow continued

CMPE I20: Computer Organization & Architecture

Olivia Weng

Logistics

- Reminder: check-in with professor is 1% of your final grade
 - Sign up [link](#)
- HW2 due this Thursday, October 1st, 11:59pm
- HW2 Quiz next Tuesday

Control flow

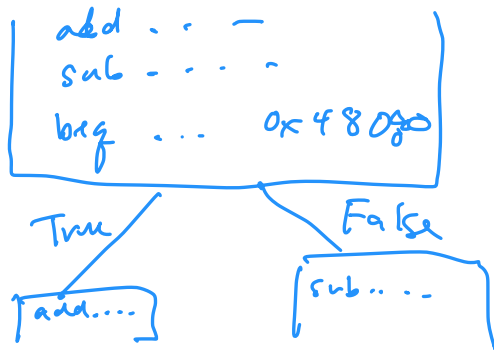
- Control flow describes **how program executes**

- Jumps
- Procedure calls — *function call*
- Conditional branch
 - if-then-else logic, loops, etc.

*↳ while(condition)
 ...
 ...*

- Control flow specifies

- Condition** (if any) under which jump or branch is taken
- If taken, the **location** to read the **next instruction** from (i.e., the **target**)



*if (condition)
 branch 1
else
 branch 2*

B-Type instruction encoding

PC-relative addressing

- Branch-type

- 3 operands

- rs1: source register 1
- rs2: source register 2
- imm: upper 12 bits of 13-bit two's complement immediate (address offset)
 - Implicit 0-bit as LSB

B-Type



↳ constant #

- op: operation code (opcode)

- funct3:

- the function combined with opcode tells machine exactly what operation to perform

beg
bne
blt

Example instruction encoding

```
0x70    beq  s0, t5, L1
0x74    add  s1, s2, s3
0x78    sub  s5, s6, s7
0x7C    lw   t0, 0(s1)
0x80 L1: addi s1, s1, -15
```

- Goto address L1 if s0 == t5
 - 13-bit immediate encodes where to branch relative to the current branch instruction
 - Address L1 = 0x80, so jump from 0x70 to 0x80 = 0x80 - 0x70 = 0x10 (+16 to our current address)
beq s0, t5, L1
→ addr offset

Example instruction encoding

- Goto address L1 if s0 == t5
 - 13-bit immediate encodes where to branch relative to the current branch instruction
 - Address L1 = 0x80, so jump from 0x70 to 0x80 = 0x80 - 0x70 = 0x10 (+16 to our current address)

beq s0, t5, L1

B-Type



imm=0000 000, rs2=30, rs1=8, funct3=0, imm=10000, op=99
 0000 000 0 0111 0100 0 000 1000 0 110 0011

00000000011101000000100001100011

0x01E40863

```
0x70      beq  s0, t5, L1
0x74      add  s1, s2, s3
0x78      sub  s5, s6, s7
0x7C      lw   t0, 0(s1)
0x80 L1:  addi s1, s1, -15
```

Example instruction encoding

```

0x70      beq  s0, t5, L1
0x74      add  s1, s2, s3
0x78      sub  s5, s6, s7
0x7C      lw   t0, 0(s1)
0x80 L1:  addi s1, s1, -15
  
```

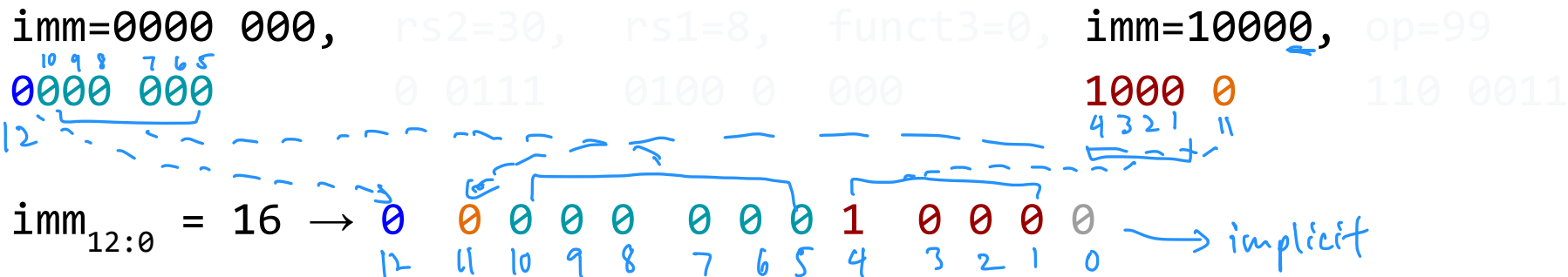
- Goto address L1 if s0 == t5
 - 13-bit immediate encodes where to branch relative to the current branch instruction
 - Address L1 = 0x80, so jump from 0x70 to 0x80 = 0x80 - 0x70 = 0x10 (+16 to our current address)

beq s0, t5, L1

B-Type

- Immediate encoding is weird

- implicit 0-bit LSB



B-Type

Practice problem

31:25	24:20	19:15	14:12	11:7	6:0
imm _{12,10:5}	rs2	rs1	funct3	imm _{4:1,11}	op
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

- How would the immediate (address offset) be encoded?

- imm_{12:0} = ?? 0000 0000 0000 1100
- imm_{12,10:5} = ?? 00000000
- imm_{4:1,11} = ?? 01100

0x70

0x74

0x78

0x7C L1:

beq s0, t5, L1

add s1, s2, s3

sub s5, s6, s7

addi s1, s1, -15

0xC bytes away

12

A. 00000000001111

B. 00000000000000

C. 010000001100

D. 000000001100

E. None of the above

imm_{12:0} =

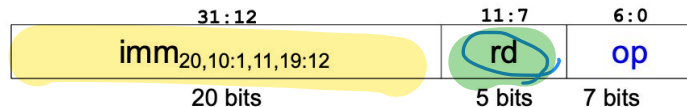
0	0	0	0	0	0	0	1	1	0	0		
12	11	10	9	8	7	6	5	4	3	2	1	0

bit idx

J-type instruction encoding

function calls

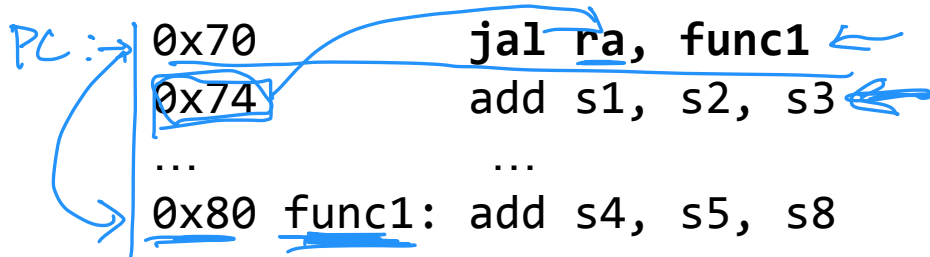
J-Type



- **Jump-type**
 - Used for jump-and-link instruction (jal)
 - jal: jump to PC-relative address and save next PC (i.e., PC +4) in destination register (typically ra)
- **2 operands**
 - **rd**: **destination** register
 - **imm**: Upper 20 bits of 21-bit immediate
 - There's an implicit 0-bit as LSB
- **op**: operation code (**opcode**)

... $x = x + i$;
 $\rightarrow \text{func1}()$;
 $\rightarrow y = x + i$;

Example instruction encoding



- jal: jump to **PC-relative address**, saving next PC (i.e., $PC + 4$) in **destination register**
 - **21-bit immediate** encodes where to branch relative to the current branch instruction
 - **imm** stores upper 20 bits with implicit 0-bit as LSB
 - Address func1 = 0x80, so jump from 0x70 to 0x80 = $0x80 - 0x70 = 0x10$ (+16 to our current address)
 - Next PC = $PC + 4 = 0x70 + 0x4 = 0x74$ (saved in ra)

Example instruction encoding

```

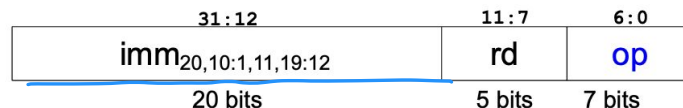
0x70      jal ra, func1
0x74      add s1, s2, s3
...
0x80 func1: add s4, s5, s8

```

- `jal`: jump to **PC-relative address**, saving next PC (i.e., **PC + 4**) in **destination register**
 - **21-bit immediate** encodes where to branch relative to the current branch instruction
 - `imm` stores upper 20 bits with implicit 0-bit as LSB
 - Address `func1 = 0x80`, so jump from `0x70` to `0x80 = 0x80 - 0x70 = 0x10` (+16 to our current address)
 - Next PC = PC + 4 = `0x70 + 0x4 = 0x74` (saved in `ra`)

`jal ra, func1`

J-Type



imm=0000 0001 0000 0000 0000
 0000 0001 0000 0000 0000

rd=1, op=111
 0000 1 110 1111

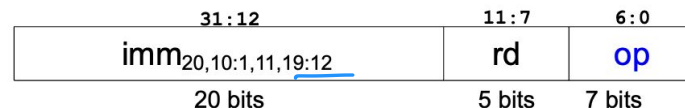
000000010000000000000000000011101111
 0x010000EF

Example instruction encoding

```
0x70      jal ra, func1
0x74      add s1, s2, s3
...
0x80 func1: add s4, s5, s8
```

- jal: jump to PC-relative address, saving next PC (i.e., $PC + 4$) in destination register
 - 21-bit immediate encodes where to branch relative to the current branch instruction
 - imm stores upper 20 bits with implicit 0-bit as LSB
 - Another weird immediate encoding

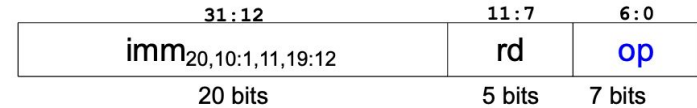
```
jal ra, func1
```



imm_{20,10:1,11,19:12} = 

[illegible]

Practice problem



- How would the immediate (address offset) be encoded?

$\circ$ $\text{imm}_{20:0} = ??$

○ $\text{imm}_{20,10:1,11,19:12} = ??$

0x70

jal ra, func1

■ ■ ■

■ ■ ■

```
0x78 func1: add s4, s5, s8
```